Data Engineering Assessment

Retail Sales Pipeline — ETL into MySQL, served through FastAPI

This is a practical exercise for graduate and junior data engineers. We want to see you take a messy real-world file, clean it with rules we give you, load it into MySQL sensibly, and put an API in front of it. **Budget about 5 hours.** It is better to finish four parts properly than to half-build six.

> **How we mark this:** we are not looking for clever code. We are looking for a pipeline that runs end to end, numbers we can reproduce, and honest notes about anything you were unsure of. If you get stuck on one rule, write down what you tried and move on — that scores better than silence.

1. The data

Download the **Online Retail II** dataset from Kaggle:

`https://www.kaggle.com/datasets/mashlyn/online-retail-ii-uci`

It is one Excel workbook with two sheets, `Year 2009-2010` and `Year 2010-2011`, covering December 2009 to December 2011. It is real transaction data from a UK wholesale giftware company, and it is not clean. Both sheets have the same columns:

Column	Meaning
`Invoice`	Invoice number. A leading **C** means the invoice was cancelled.
`StockCode`	Product code.
`Description`	Product name, typed by a person.
`Quantity`	Units on the line. Can be negative.
`InvoiceDate`	When the transaction happened.
`Price`	Price per unit, in GBP.
`Customer ID`	Customer reference. Often blank.
`Country`	Where the customer is based.

Treat both sheets as one history. Do not delete either of them, and do not edit the workbook by hand.

Files we give you

Three CSVs come with this assessment. Put them in a `data/reference/` folder. **There are no APIs to call** — everything you need is in the dataset and these files.

File	What it is
`fx_rates_gbp_usd.csv`	Daily GBP→USD rates. Weekends and holidays are missing, because the market is closed.
`country_region.csv`	Maps every `Country` value in the data to a tidy country name and a region.
`non_product_codes.csv`	Stock codes that are postage, fees or adjustments rather than real products.

2. Part 1 — Load the raw data

- Read both sheets into Python.
- Save the rows **exactly as they arrive**, before you change anything, either into MySQL tables named `stg_*` or as files under `data/raw/`. This is your audit trail.
- The whole pipeline should run from a single command, for example `python -m pipeline.run`.

- Running it twice must not double your row counts.

3. Part 2 — Clean it up

Apply these eight rules in order. Where a rule does not say exactly what to do, make a sensible choice and write it down in `ASSUMPTIONS.md` next to the rule number. **Never delete a row silently** — if you drop or change something, record it.

R1 — Remove exact duplicates. Some rows are repeated identically, right down to the timestamp. Keep one of each and report how many you removed.

R2 — Cancellations. An invoice number starting with **C** is a cancellation. Careful: **a negative quantity is not the same thing as a cancellation.** Some rows have a negative quantity on a perfectly normal invoice number — those are stock write-offs, not customer returns. Count and flag the two separately.

This is the rule most people get wrong. Check before you assume.

R3 — Customers without an ID. Roughly a fifth of rows have a blank `Customer ID`. These were cash sales over the counter. Keep the rows — they are real sales — but group them under a single placeholder customer, and leave them out of anything counted per customer.

R4 — Lines that are not products. Some stock codes are postage, bank charges or manual adjustments. They are listed in `non_product_codes.csv`. Keep the rows but mark them, and leave them out of product revenue. Match the codes ignoring upper and lower case.

R5 — Odd prices. Some lines are priced at zero and a very small number are priced below zero. Treat them differently: zero-price lines are free samples, so keep them but do not count them as revenue; negative prices are mistakes, so reject them and log them.

R6 — Line total. `gross_gbp = Quantity × Price`. Store money as `DECIMAL`, never as a float, and round to 2 decimal places only at the very end.

R7 — Convert to US dollars. Use `fx_rates_gbp_usd.csv` to convert each line to USD using the rate **for that invoice date**. That file has no weekend rows, so when there is no rate for a date, use **the most recent rate before it** — never a later one. Save both the rate you used and the date that rate came from.

Hint: a plain join will quietly lose every weekend row. Check your row count before and after.

R8 — Country and region. Map the `Country` column through `country_region.csv`. Two values map to **Unknown** rather than to a real country. Keep them as Unknown; do not drop them and do not guess.

4. Part 3 — Model it in MySQL

Design a small star schema. We are not expecting anything elaborate — something like this is fine:

- Dimensions: `dim_customer`, `dim_product`, `dim_country`, `dim_date`.
- A fact table of invoice lines, with foreign keys to those dimensions.
- A `data_quality_issue` table where every row you rejected or corrected gets logged: which rule, which row, and what was wrong.

We will be looking for sensible column types (money as `DECIMAL`, dates as `DATE`), primary keys, and an index or two where a query would obviously need one.

5. Part 4 — Serve it with FastAPI

Build a small FastAPI service that reads from MySQL. Let the database do the grouping and summing — do not pull every row into Python and add it up there.

Endpoint	What it returns
`GET /health`	A simple OK, plus whether the database connection works.
`GET /customers/{customer_id}/summary`	That customer's country and region, first and last order date, number of invoices, and total spend in GBP and USD.
`GET /products/top?month=YYYY-MM&limit=10`	The best-selling products that month by revenue in USD.

`GET /kpis/monthly?start=YYYY-MM&end=YYYY-MM&region=`	Per month: number of invoices, units sold, revenue in GBP and USD. `region` is optional.
`GET /data-quality/summary`	How many issues you logged, grouped by rule.

Handle the obvious mistakes properly: a customer who does not exist should return **404**, and a badly formatted month should return **422** rather than crashing. Put a sensible maximum on `limit`.

6. What to submit

A Git repository (a link or a zip) containing:

- `README.md` — how to set it up, how to run the pipeline, how to start the API, and a couple of example `curl` commands.
- `ASSUMPTIONS.md` — every decision you made where the rules left you a choice, listed by rule number.
- `RESULTS.md` — the numbers listed below.
- Your `CREATE TABLE` statements.
- At least two unit tests. We suggest testing R2 and R7, which are the fiddliest.

Numbers to put in RESULTS.md

- How many rows you read in total, and how many R1 removed as duplicates.
- How many cancellation lines, and separately how many negative-quantity lines that were **not** cancellations (R2).
- How many rows you excluded under R4, and how many under R5.
- How many distinct customers you ended up with, not counting the placeholder from R3.
- Total revenue in GBP and USD across the whole period.
- Revenue in GBP and USD for **December 2010**.
- Your top 5 products by USD revenue for **March 2011**.
- A count of data-quality issues per rule.

7. How we mark it

Area	Points	What we are looking for
Rules applied correctly	30	Your RESULTS.md numbers match ours. R2 and R7 carry the most weight.
Handling the messy data	20	Problems found, logged and explained. Nothing dropped in silence.
Database design and load	20	Sensible tables and types, keys, and a rerun that does not duplicate.
The API	15	It runs, it returns the right numbers, it aggregates in SQL, and it handles bad input.
Code and documentation	15	Readable, organised, and we can run it from your README alone.

8. Ground rules

- Python, MySQL and FastAPI. Any libraries you like.
- Using AI tools is fine, and we would rather you said where you used them. We will ask you to walk us through your code, so make sure you can explain every part of it.
- If something here looks wrong or contradicts itself, make a decision, write it in `ASSUMPTIONS.md`, and carry on. Spotting it earns marks.
- If you run out of time, say so in the README and tell us what you would have done next. That is a perfectly good answer.